

DESARROLLO DE INTERFACES · 0488

Actividad guiada — Ionic + Angular Standalone: consumo de API REST

App **dam2-productos** · dummyjson.com/products · GitHub + Vercel

Alumno Jorge González Gómez

Curso 2º DAM — Universidad Nebrija

Fecha 06-10-2026 (versión 1, entrega parcial)

GitHub github.com/JorgeGonzalezDI/Desarrollo-Interfaces-DAM-2/tree/main/03-API-REST-Productos

Ramas: `desarrollo` (trabajo) y `main` (validado)

Vercel <https://dam2-productos-jorgegonzalez.vercel.app>

Nota del alumno: esta es una primera versión. Estoy repasando los conceptos de base de Angular (componentes, data binding, signals) y de Observables antes de completar el resto de peticiones. Iré actualizando el proyecto en GitHub y este documento a lo largo de la semana.

1. Estado de la entrega

Nº	Petición	Estado
—	Base de la actividad guiada: modelo, servicio con HttpClient, rutas, página Inicio, tabla de productos y estados de carga / error / reintento	Hecho
1	Dimensiones de ancho y alto en la tabla	Hecho
2	Stock valorado = unidades × precio – descuento aplicable	Hecho
3	Botón para regresar a la página Inicio	Hecho
4	Página About con reseña de la empresa y enlace a GitHub	Pendiente
5	Troubleshooting (al menos dos ejemplos)	Hecho (apartado 5)
6	Reto: paginación	Pendiente
7	¿Por qué existe <code>app.config.ts</code> ?	Hecho (apartado 3)
8	Flujo de ramas <code>desarrollo</code> → <code>main</code> y Vercel desde <code>main</code>	Hecho (apartado 4)
9	Botón modo oscuro	Pendiente
10	Reto: vista más visual que una tabla (cards / dashboard)	Pendiente

2. Código más relevante comentado

2.1 Modelo — `models/product.model.ts`

Describe la respuesta de la API. Se añade `Dimensions` para la petición 1. La API no devuelve un array directo sino un objeto que lo envuelve, por eso existe `ProductsResponse`.

```
export interface Dimensions { width: number; height: number; depth: number; }

export interface Product {
  id: number; title: string; description: string; category: string;
  price: number; discountPercentage: number; rating: number; stock: number;
  brand?: string;           // opcional: algunos productos no traen marca
  thumbnail: string;
  dimensions: Dimensions;   // petición nº 1
}

export interface ProductsResponse { products: Product[]; total: number; skip: number; limit:
number; }
```

2.2 Servicio — `services/product.service.ts`

Toda la comunicación HTTP está en el servicio, nunca en la página. `HttpClient` se obtiene por inyección de dependencias con `inject()` y devuelve un **Observable**: la petición no se envía hasta que alguien se suscribe, por eso no se usa `async/await`.

```

@Injectable({ providedIn: 'root' }) // una instancia para toda la app
export class ProductService {
  private http = inject(HttpClient); // DI moderna (equivale al constructor)
  private apiUrl = 'https://dummyjson.com/products';

  getProducts(): Observable<ProductsResponse> {
    return this.http.get<ProductsResponse>(this.apiUrl); // GET tipado con genérico
  }
}

```

2.3 Página Productos — `productos.page.ts`

Los datos se guardan en **signals** (ver troubleshooting 5.2). En `subscribe` se tratan los dos casos: `next` (llega la respuesta) y `error`. El método `stockValorado()` resuelve la petición 2.

```

products = signal<Product[]>([]);
total = signal(0);
loading = signal(false);
error = signal('');

loadProducts(): void {
  this.loading.set(true);
  this.error.set('');
  this.productService.getProducts().subscribe({ // aquí se envía la petición
    next: (response: ProductsResponse) => {
      this.products.set(response.products); // la lista va DENTRO de la respuesta
      this.total.set(response.total);
      this.loading.set(false);
    },
    error: (err) => {
      console.error(err);
      this.error.set('No se han podido cargar los productos.');
```

```

<!-- Petición nº 3: botón para regresar a inicio (necesita RouterLink en imports) -->
<ion-button routerLink="/inicio" fill="outline">Volver a inicio</ion-button>

@if (loading()) { <ion-spinner></ion-spinner> ... }
@if (error()) { ... <ion-button (click)="loadProducts()">Reintentar</ion-button> }
@if (!loading() && !error()) {
  ...
  @for (product of products(); track product.id) {
    <tr>
      ...
      <td>{{ product.brand || 'Sin marca' }}</td>
      <td>{{ product.price | currency: 'EUR' }}</td>
      <!-- Petición nº 1: ancho x alto -->
      <td>{{ product.dimensions.width | number: '1.0-2' }} × {{ product.dimensions.height |
number: '1.0-2' }}</td>
      <!-- Petición nº 2: stock valorado -->
      <td>{{ stockValorado(product) | currency: 'EUR' }}</td>
    </tr>
  }
}

```

2.5 Rutas — `app.routes.ts`

```

{ path: '', redirectTo: 'inicio', pathMatch: 'full' },
{ path: 'inicio', loadComponent: () => import('./pages/inicio/inicio.page').then(m =>
m.InicioPage) },
{ path: 'productos', loadComponent: () => import('./pages/productos/productos.page').then(m =>
m.ProductosPage) },
{ path: '**', redirectTo: 'inicio' } // ruta desconocida → inicio (va la última)

```

3. Petición 7 — ¿Por qué existe `app.config.ts`?

En Angular standalone ya no hay `AppModule`. Lo que antes se declaraba en el `providers` del módulo (router, HttpClient, configuración de Ionic...) ahora se agrupa en un objeto `ApplicationConfig`, que se pasa a `bootstrapApplication()` al arrancar. Ese objeto vive en `app.config.ts`: es la **configuración global** de la aplicación y el sitio donde se registran los `providers` que la inyección de dependencias podrá entregar a cualquier componente o servicio.

```

// app.config.ts
export const appConfig: ApplicationConfig = {
  providers: [
    { provide: RouteReuseStrategy, useClass: IonicRouteStrategy },
    provideIonicAngular(),
    provideRouter(routes, withPreloading(PreloadAllModules), withComponentInputBinding()),
    provideHttpClient(), // necesario para poder inyectar HttpClient en el servicio
  ],
};

// main.ts
bootstrapApplication(AppComponent, appConfig);

```

La plantilla *blank* de Ionic no trae este fichero (pone los providers directamente en `main.ts`); lo he creado para separar la configuración del arranque, como indica la actividad (ver troubleshooting 5.3).

4. Petición 8 — Flujo de ramas: desarrollo → main

`main` es la rama "buena": solo recibe código revisado y validado, y es la que publica Vercel. El trabajo diario se hace en `desarrollo`; cuando una versión funciona, se pasa a `main` con un **merge**.

```
git checkout desarrollo          # 1. trabajar siempre en desarrollo
git add .
git commit -m "Descripción del cambio"
git push origin desarrollo      # 2. subir la rama de trabajo

# 3. cuando está probado y validado:
git checkout main               # cambiar a main
git merge desarrollo            # traer a main los cambios de desarrollo
git push origin main            # subir main → Vercel despliega automáticamente
git checkout desarrollo         # 4. volver a desarrollo para seguir trabajando
```

Alternativa en GitHub: abrir un **Pull Request** de `desarrollo` a `main` y pulsar *Merge*. En Vercel, *Settings* → *Git* → *Production Branch* = `main`; los pushes a `desarrollo` generan solo despliegues de vista previa (*Preview*). En este proyecto ya se ha trabajado así: los commits de la actividad se hicieron en `desarrollo` y se pasaron a `main` con `git merge`.

5. Petición 5 — Troubleshooting (resolución de incidencias)

5.1 Error al compilar: no se encuentra '@ionic/angular/standalone'

Síntoma: al copiar los imports del enunciado (`import { IonHeader, ... } from '@ionic/angular/standalone'`) el proyecto no compila.

Causa: el proyecto se ha creado con Ionic 9, donde esa ruta ya no existe; en Ionic 9 los componentes standalone se importan directamente de '@ionic/angular'. Se comprobó revisando los `exports` de `node_modules/@ionic/angular/package.json`.

Solución: cambiar todos los imports a `from '@ionic/angular'`.

5.2 La página se queda en "Cargando productos..." para siempre

Síntoma: con el código del enunciado (`loading = false`, `products: Product[] = []` y asignaciones dentro del `subscribe`) la petición llega bien (se ve en F12 → Network), pero la pantalla no se actualiza.

Causa: Angular 22 funciona sin `zone.js` (*zoneless*): ya no detecta automáticamente los cambios hechos en propiedades normales dentro de un callback asíncrono, así que no vuelve a pintar la vista.

Solución: guardar el estado en **signals** (`signal() + .set()`), que avisan a Angular de cada cambio, y leerlos en la plantilla como `loading()`, `products()`. Otra alternativa válida es el pipe `async`.

5.3 app.config.ts no existe en el proyecto generado

Síntoma: el enunciado pide añadir `provideHttpClient()` en `src/app/app.config.ts`, pero la plantilla *blank* de Ionic no crea ese fichero.

Solución: crear `app.config.ts` con un `ApplicationConfig` que contiene todos los providers (los que traía `main.ts` + `provideHttpClient()`) y cambiar `main.ts` a `bootstrapApplication(AppComponent, appConfig)`.

5.4 Error 404 al recargar `/productos` en Vercel

Síntoma: navegando desde Inicio funciona, pero al recargar o abrir directamente `/productos` Vercel devuelve 404.

Causa: es una SPA: solo existe `index.html` y las rutas las resuelve Angular en el navegador; Vercel busca un fichero `/productos` que no existe.

Solución: añadir `vercel.json` con `{"rewrites": [{"source":"/(.*)","destination":"/index.html"}]}` y `"engines": {"node": "24.x"}` en `package.json` (Angular 22 necesita Node ≥ 22.22).

6. Pendiente y conceptos que estoy repasando

- **Pendiente:** página About (4), paginación (6), botón de modo oscuro (9) y vista con cards/dashboard (10).
- **Repasando:** data binding (interpolación, property, event y two-way binding), signals y la diferencia entre Promise y Observable (`subscribe` , pipe `async`). Cuando los tenga claros completaré las peticiones pendientes y actualizaré este documento y el repositorio.

7. Enlaces

- GitHub: <https://github.com/JorgeGonzalezDI/Desarrollo-Interfaces-DAM-2/tree/main/03-API-REST-Productos>
- Vercel: <https://dam2-productos-jorgegonzalez.vercel.app>
- API: <https://dummyjson.com/products>